

파이썬 프로그래밍 함수(Function) 수업

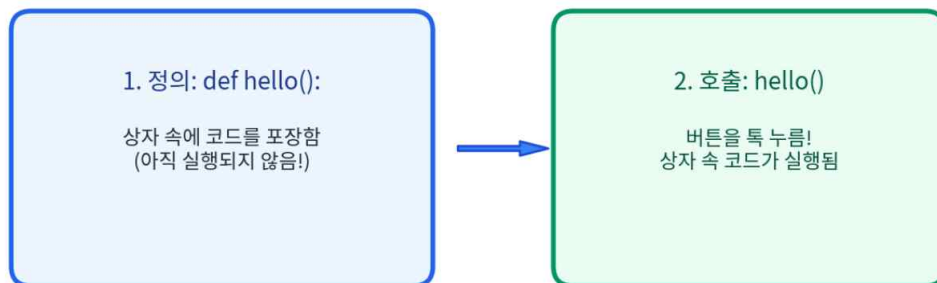
핵심 흐름: 정의하기 → 호출하기 → 값 전달하기 → 결과 돌려받기 → 함수로 문제 나누기

[1차시] 함수 정의와 호출 - 함수를 처음 만나다

학습 목표: 함수가 왜 필요한지 설명할 수 있고, def를 사용하여 함수를 정의한 뒤 함수 이름으로 호출할 수 있다.

1. 개념 이야기: 주문 즉시 움직이는 마법 상자

자주 반복해서 해야 하는 일이 있다면, 그때마다 똑같은 코드를 길게 적는 것은 지치고 힘든 일입니다. 파이썬에서는 자주 쓰는 동작을 마법 상자에 차곡차곡 담아두고 이름표를 붙여둘 수 있습니다. 이것을 함수 정의(def)라고 합니다. 그리고 상자에 붙은 빨간 버튼을 톡 누르면 상자 안의 코드가 즉시 동작하는데, 이것을 함수 호출(call)이라고 합니다. 상자를 만들어두기만 하면 아무 일도 일어나지 않으니, 꼭 이름을 불러서 실행시켜 주어야 합니다!



2. 설명용 핵심 예제

```
def hello():  
    print("안녕하세요!")  
    print("파이썬 함수 수업입니다.")  
  
hello()  
hello()
```

3. 코드 한 줄씩 살펴보기

- **def hello():** hello라는 이름의 함수를 새로 정의합니다.
- **들여쓰기(Tab)** 콜론(:) 다음 줄부터 함수가 실행할 내용을 들여쓰기하여 상자 속에 넣습니다.
- **정의와 실행의 차이** 함수를 정의하기만 하면 화면에 아무 일도 일어나지 않습니다.
- **hello()** hello()처럼 소괄호를 붙여 호출해야 상자 안의 코드가 비로소 실행됩니다.

4. 실습 도전 과제

- **실습 1.** 함수 이름을 `welcome()`으로 바꾸어 보기
- **실습 2.** 함수 안에서 자신의 이름과 좋아하는 과목을 출력해 보기
- **실습 3.** 함수를 3번 호출했을 때 결과가 어떻게 출력되는지 확인해 보기

5. 수업 마무리 확인 퀴즈

- Q. 함수를 만드는 파이썬 키워드는 무엇인가요?

정답 확인: `def` (`define`의 약자)

- Q. 함수는 정의하는 순간 바로 화면에 실행되나요?

정답 확인: 실행되지 않으며, 호출해야 실행됩니다.

- Q. 함수 이름 `hello`와 호출 코드 `hello()`의 결정적 차이는 무엇인가요?

정답 확인: `hello`는 함수의 이름 자체를 가리키며, `()`가 붙어야 실제로 실행됩니다.

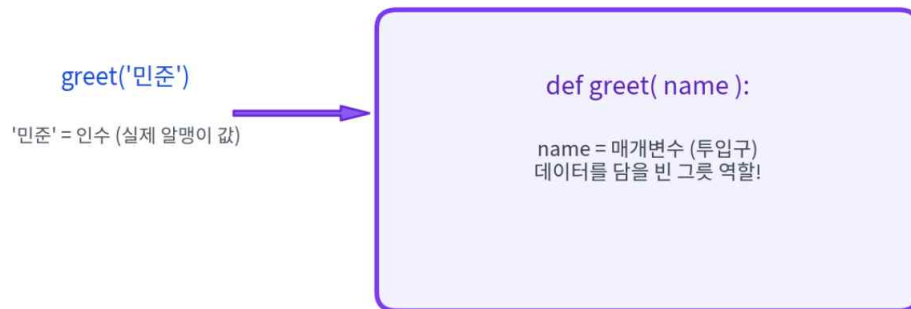
오늘의 핵심 한 문장 : 함수는 코드를 이름으로 묶고, 호출하여 실행한다.

[2차시] 매개변수와 인수 - 함수에 값을 전달하기

학습 목표: 매개변수와 인수의 의미를 구분할 수 있고, 호출할 때 전달한 값을 함수 내부에서 사용할 수 있다.

1. 개념 이야기: 동전 투입구가 있는 만능 자판기

언제나 똑같은 말만 하는 앵무새 상자보다, 내가 건네주는 말에 맞춰 다르게 대답하는 상자가 훨씬 똑똑하겠죠? 함수 상자 뚜껑에 투입구(슬롯)를 만들어두면 외부에서 원하는 재료를 쏙 집어넣을 수 있습니다. 이때 함수 정의에서 값을 받아들 빈 그릇 변수를 매개변수(parameter)라고 하고, 함수를 부를 때 실제로 쏙 집어넣는 알맹이 값을 인수(argument)라고 부릅니다.



2. 설명용 핵심 예제

```
def greet(name):  
    print(name, "님, 안녕하세요!")  
  
greet("민준")  
greet("서연")
```

3. 코드 한 줄씩 살펴보기

- **def greet(name):** name은 외부에서 들어올 손님 데이터를 담기 위해 마련한 매개변수(빈 그릇)입니다.
- **print(name, ...)** 전달받은 name 그릇에 담긴 값을 이용해 맞춤 인사를 만듭니다.
- **greet('민준')** 민준이라는 실제 값(인수)을 던져주어 민준이 전용 인사를 출력합니다.
- **greet('서연')** 서연이라는 인수를 던져주면 같은 함수라도 서연이 전용 결과가 나옵니다.

4. 실습 도전 과제

- **실습 1.** 이름과 학년을 받아 소개하는 `introduce(name, grade)` 함수 만들기
- **실습 2.** 숫자 하나를 받아 그 수의 2배를 출력하는 `double(n)` 함수 만들기

5. 수업 마무리 확인 퀴즈

- Q. 매개변수는 함수 정의와 호출 중 어디에 등장하나요?

정답 확인: 함수를 정의(def)할 때 괄호 안에 등장합니다.

- Q. greet('지우')에서 지우는 매개변수인가요, 인수인가요?

정답 확인: 함수를 호출할 때 실제로 전달하는 값이므로 인수(argument)입니다.

오늘의 핵심 한 문장 : 매개변수는 함수가 값을 받아 사용하는 통로이다.

[3차시] return - 함수의 결과를 돌려받기

학습 목표: print와 return의 차이를 명확히 설명할 수 있고, 함수가 계산한 값을 return으로 반환하여 활용할 수 있다.

1. 개념 이야기: 전광판(print) vs 계산 결과 전달(return)

마트 계산원이 계산기 화면을 손님에게 눈으로만 보여주고(print), 영수증 금액을 다음 카드 결제 기계에는 아무것도 넘겨주지 않는다면 결제를 마칠 수 없겠죠? print는 단순히 컴퓨터 모니터에 글자를 찍고 사라지는 전광판일 뿐입니다. 반면에 return은 계산된 최종 결과물을 함수 밖으로 꺼내어 호출한 사람의 손에 쥐여줍니다. 그 결과물을 변수에 담아 다른 계산이나 다음 함수에 자유롭게 연결할 수 있습니다!



2. 설명용 핵심 예제

```
def add(a, b):  
    result = a + b  
    return result  
  
answer = add(10, 20)  
print(answer)  
print(add(3, 5) * 2)
```

3. 코드 한 줄씩 살펴보기

- **return result** 계산된 result 값을 함수 밖으로 즉시 전달하고 함수를 끝마칩니다.
- **answer = add(10, 20)** add 함수가 돌려준 값(30)이 answer 변수에 안전하게 저장됩니다.
- **print(add(3, 5) * 2)** add(3, 5)가 반환한 8에 바로 2를 곱해 16을 출력하는 연쇄 계산이 가능해집니다.

4. 실습 도전 과제

- **실습 1.** 두 수의 곱을 반환하는 multiply(a, b) 함수 만들기
- **실습 2.** 원의 반지름을 받아 원의 넓이를 반환하는 circle_area(r) 함수 만들기 ($\pi = 3.14$)

5. 수업 마무리 확인 퀴즈

- Q. `print`로 출력한 값과 `return`으로 돌려받은 값은 어떻게 다른가요?

정답 확인: `print`는 모니터에 표시만 되고 메모리 변수에 저장되지 않지만, `return`은 변수에 담아 재활용할 수 있습니다.

- Q. `return`문 바로 아래에 작성된 코드는 실행될까요?

정답 확인: 실행되지 않습니다. `return`을 만나는 순간 함수는 즉시 종료됩니다.

오늘의 핵심 한 문장 : `return`은 함수의 결과를 함수 밖으로 돌려준다.

[4차시] 여러 개의 매개변수 - 계산 함수 만들기

학습 목표: 두 개 이상의 매개변수를 유연하게 다루고, 위치 인수와 키워드 인수의 전달 방식을 구분할 수 있다.

1. 개념 이야기: 줄 서서 들어가는 순서 vs 이름표 달고 들어가는 배송

재료가 많이 들어가는 요리를 할 때 소금, 설탕, 간장을 순서대로 정확히 넣어야 할 때가 있습니다. 함수를 호출할 때 값을 적은 순서대로 첫 번째, 두 번째, 세 번째 매개변수에 들어가는 방식을 위치 인수라고 합니다. 하지만 매개변수가 많아지면 헷갈릴 수 있겠죠? 그럴 때는 `school='한솔고'`처럼 매개변수 이름표를 직접 붙여서 전달할 수도 있습니다. 이것을 키워드 인수라고 부르며 순서가 바뀌어도 안전합니다!

```
def student_info( name, age, school ):
```

1) 위치 인수: `student_info('지민', 17, '한솔고')` -> 적은 순서대로 1:1 매칭

2) 키워드 인수: `student_info(school='한솔고', name='지민', age=17)` -> 이름표가 있어 순서 상관없음!

2. 설명용 핵심 예제

```
def student_info(name, age, school):  
    print("이름:", name)  
    print("나이:", age)  
    print("학교:", school)  
  
# 1) 위치 인수 방식 (순서가 일치해야 함)  
student_info("지민", 17, "한솔고")  
  
# 2) 키워드 인수 방식 (이름표를 붙여 순서 상관 없음)  
student_info(school="한솔고", name="지민", age=17)
```

3. 코드 한 줄씩 살펴보기

- `student_info('지민', 17, '한솔고')` 첫 번째 호출은 작성된 위치에 따라 `name`, `age`, `school`로 순서대로 매칭됩니다.
- `student_info(school=...)` 두 번째 호출은 매개변수 이름을 직접 지정했으므로 순서를 뒤바꾸어도 정확히 들어갑니다.

4. 실습 도전 과제

- **실습 1.** 가로와 세로를 받아 직사각형의 넓이를 반환하는 `rectangle(w, h)` 함수 만들기
- **실습 2.** 국어, 영어, 수학 점수를 받아 3과목의 평균을 반환하는 `average(kor, eng, math)` 함수 만들기

5. 수업 마무리 확인 퀴즈

- Q. 인수의 순서를 바꾸었을 때 결과가 달라질 수 있는 이유는 무엇인가요?
정답 확인: 위치 인수는 오직 전달 순서에 따라 매개변수에 대입되기 때문입니다.
- Q. 키워드 인수를 사용했을 때의 가장 큰 장점은 무엇인가요?
정답 확인: 매개변수의 순서를 일일이 외우지 않아도 되고, 코드의 가독성이 높아집니다.

오늘의 핵심 한 문장 : 함수는 필요한 만큼 여러 값을 전달받을 수 있다.

[5차시] 기본값 매개변수 - 값을 생략해도 되는 함수

학습 목표: 자주 사용하는 값을 기본값 매개변수로 지정하고, 생략 여부에 따른 동작 원리를 설명할 수 있다.

1. 개념 이야기: 단골 손님을 위한 자동 기본 옵션

카페에서 아메리카노를 시킬 때 별말이 없으면 기본적으로 차가운 아이스로 나오는 곳이 있습니다. 굳이 말하지 않아도 알아서 정해진 기본 옵션으로 동작하는 것입니다! 함수를 정의할 때 `message='안녕하세요'`처럼 미리 기본값을 채워두면, 호출할 때 깜빡 잊거나 생략해도 기본값이 든든하게 작동합니다. 물론 특별한 메시지를 적어주면 기본값을 덮어쓰고 새로운 메시지가 나옵니다.

```
def greet( name, message = '안녕하세요' ):
```

호출 1: `greet('민수')` ==> 인수를 생략하면 정해진 기본값('안녕하세요') 사용

호출 2: `greet('민수', '반갑습니다')` ==> 새로운 값을 넣으면 기본값을 덮어씀

2. 설명용 핵심 예제

```
def greet(name, message="안녕하세요"):
    print(name, "님,", message)

greet("민수")
greet("민수", "반갑습니다")
```

3. 코드 한 줄씩 살펴보기

- `message='안녕하세요'` 두 번째 매개변수를 생략하면 안녕하세요를 기본값으로 사용하도록 선언합니다.
- `greet('민수')` message 인수를 생략했으므로 기본값 안녕하세요가 자동 적용됩니다.
- `greet('민수', '반갑습니다')` 새로운 값을 전달하면 기본값 대신 전달된 반갑습니다가 사용됩니다.

4. 실습 도전 과제

- **실습 1.** price와 discount=10을 받아 10% 할인을 기본으로 계산하는 함수 만들기
- **실습 2.** name과 times=1을 받아 이름을 times번 반복 출력하는 함수 만들기

5. 수업 마무리 확인 퀴즈

- Q. 기본값 매개변수를 유용하게 사용하는 이유는 무엇인가요?

정답 확인: 자주 쓰는 옵션을 기본 설정해 두어 함수 호출을 훨씬 간결하고 편리하게 만들기 위함입니다.

- Q. 기본값이 없는 매개변수를 기본값이 있는 매개변수 뒤에 두면 왜 문법 오류가 날까요?

정답 확인: 호출할 때 어떤 매개변수를 생략했는지 컴퓨터가 위치상 판별할 수 없기 때문입니다.

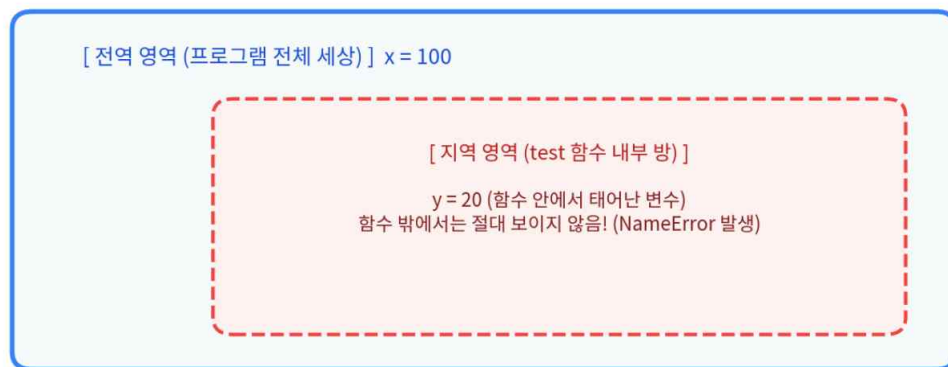
오늘의 핵심 한 문장 : 기본값을 사용하면 함수를 더 편리하고 유연하게 호출할 수 있다.

[6차시] 지역 변수와 전역 변수 - 변수는 어디까지 사용할 수 있을까?

학습 목표: 함수 내부의 지역 변수와 외부의 전역 변수의 유효 범위를 구분하고 올바른 데이터 전달 습관을 기른다.

1. 개념 이야기: 내 방 안의 물건(지역) vs 거실의 물건(전역)

내 방 책상 위에 놓인 일기장은 내 방 안에서만 볼 수 있고 집 밖에서는 볼 수 없습니다. 반면에 거실 벽에 걸린 시계는 온 가족이 함께 볼 수 있죠. 함수 안에서 def로 새로 만들어진 변수는 그 함수라는 방 안에서만 살아있는 지역 변수(Local)입니다. 함수가 끝나면 방 문이 닫히듯 사라져 버립니다! 밖에서 억지로 꺼내 쓰려 하면 NameError가 발생하므로, 필요한 값은 꼭 return을 통해 밖으로 전달해야 합니다.



2. 설명용 핵심 예제

```
x = 100 # 전역 변수 (함수 밖 어디서나 접근 가능)

def test():
    y = 20 # 지역 변수 (test 함수 방 안에서만 존재)
    print("함수 안 x:", x)
    print("함수 안 y:", y)

test()
print("함수 밖 x:", x)
# print(y) # NameError 발생! y는 함수 밖에서 사용할 수 없음
```

3. 코드 한 줄씩 살펴보기

- **x = 100** 함수 밖에서 선언된 전역 변수로 어디서나 읽을 수 있습니다.
- **y = 20** test() 내부에서 태어난 지역 변수이므로 함수 내부에서만 존재합니다.
- **# print(y)** 함수가 끝나는 순간 y 변수는 사라지므로 함수 밖에서 호출하면 NameError가 발생합니다.

4. 실습 도전 과제

- **실습 1.** 함수 안에서 만든 score 변수를 함수 밖에서 출력해 보고 발생하는 NameError 직접 확인하기

- **실습 2.** 전역 변수를 쓰지 않고, 매개변수와 return을 활용해 깔끔한 계산 프로그램 다시 작성해 보기

5. 수업 마무리 확인 퀴즈

- Q. 지역 변수는 코드의 어느 영역에서만 사용할 수 있나요?

정답 확인: 자신이 정의된 해당 함수 블록(내부) 안에서만 유효합니다.

- Q. 함수 간 데이터 전달 시 전역 변수 남용보다 매개변수와 return을 권장하는 이유는?

정답 확인: 어디서 변수가 바뀌었는지 추적하기 어렵고 다른 코드와 이름이 충돌하는 버그를 방지하기 위해서입니다.

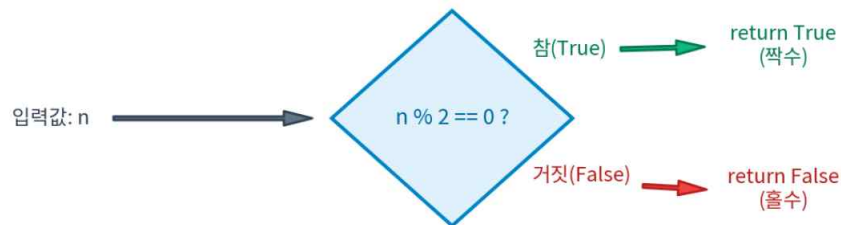
오늘의 핵심 한 문장 : 함수 안에서 만든 지역 변수는 기본적으로 함수 밖에서 사용할 수 없다.

[7차시] 함수 + 조건문 - 판별 함수 만들기

학습 목표: 함수 내부에 if문을 결합하여, 조건에 따라 True/False나 다른 결과값을 반환하는 판별 함수를 제작한다.

1. 개념 이야기: 질문에 O/X로 똑 부러지게 답하는 판사 함수

판사님에게 이 행동이 규칙에 맞습니까?라고 물어보면 명확하게 그렇다(True) 또는 아니다(False)로 판결을 내려줍니다. 함수 안에 if 조건문을 넣어두면 질문 하나를 명쾌하게 해결해 주는 판별 전용 함수를 만들 수 있습니다. 짝수인지 검사하는 함수, 합격인지 검사하는 함수를 따로 만들어두면 메인 프로그램 코드가 동화책처럼 술술 읽히게 됩니다!



2. 설명용 핵심 예제

```
def is_even(n):
    if n % 2 == 0:
        return True
    else:
        return False

number = int(input("숫자 입력: "))

if is_even(number):
    print("짝수입니다.")
else:
    print("홀수입니다.")
```

3. 코드 한 줄씩 살펴보기

- **if n % 2 == 0:** 전달받은 숫자를 2로 나눈 나머지가 0인지 조건을 검사합니다.
- **return True / return False** 조건에 따라 참/거짓의 불리언(Boolean) 값을 깔끔하게 반환합니다.
- **if is_even(number):** 메인 코드에서 복잡한 수식 대신 마치 영어 문장처럼 직관적으로 조건을 걸 수 있습니다.

4. 실습 도전 과제

- **실습 1.** 점수를 받아 60점 이상이면 True를 반환하는 `is_pass(score)` 함수 만들기
- **실습 2.** 나이를 받아 청소년/성인을 문자열로 반환하는 `age_group(age)` 함수 만들기

5. 수업 마무리 확인 퀴즈

- Q. 판별 함수가 True/False를 반환하면 메인 코드에서 어떤 점이 편리해지나요?

정답 확인: `if is_even(n):` 처럼 `if` 조건문의 조건식 자리에 바로 결합할 수 있어 코드가 깔끔해집니다.

- Q. 조건 검사 로직을 함수로 따로 분리했을 때의 장점은?

정답 확인: 동일한 판별 기준이 프로그램 여러 곳에서 쓰일 때 기준 수정이 한곳에서 가능해집니다.

오늘의 핵심 한 문장 : 조건문을 함수에 넣으면 입력값을 판별하는 기능을 만들 수 있다.

[8차시] 함수 + 반복문 - 여러 데이터를 처리하기

학습 목표: 리스트를 함수의 매개변수로 전달받아 내부 for 문으로 일괄 누적 및 반복 처리하는 함수를 작성한다.

1. 개념 이야기: 상자에 담긴 시험지를 채점하는 채점기

시험지가 1장만 있는 게 아니라 학급 30명의 시험지가 상자(리스트)에 가득 담겨 있습니다. 이 상자를 통째로 채점기 함수에 밀어 넣으면, 함수 속 for 반복문이 시험지를 한 장씩 쓱쓱 꺼내며 점수를 누적 합산해 줍니다. 리스트를 함수에 전달하는 법을 배우면 수백, 수만 개의 빅데이터도 버튼 한 번으로 요약하고 분석할 수 있습니다!



2. 설명용 핵심 예제

```
def total(numbers):  
    result = 0  
    for n in numbers:  
        result += n  
    return result  
  
scores = [80, 90, 75, 100]  
print("합계:", total(scores))
```

3. 코드 한 줄씩 살펴보기

- **def total(numbers):** numbers 매개변수에 리스트 전체가 통째로 전달됩니다.
- **for n in numbers:** 리스트 안의 원소(점수)들을 하나씩 순서대로 꺼내어 n에 담습니다.
- **result += n** 꺼낸 점수를 result 저금통에 계속해서 누적 합산합니다.
- **return result** 모든 반복이 끝나면 완성된 총점을 반환합니다.

4. 실습 도전 과제

- **실습 1.** 리스트에서 80점 이상인 우수 점수의 개수를 세어 반환하는 count_high(scores) 함수 만들기
- **실습 2.** 내장 max() 함수를 쓰지 않고, for 문으로 리스트의 최댓값을 직접 찾아 반환하는 find_max(numbers) 함수 만들기

5. 수업 마무리 확인 퀴즈

- Q. 숫자나 문자열뿐만 아니라 리스트 자료형도 함수의 인수로 전달할 수 있나요?

정답 확인: 네, 리스트, 튜플, 딕셔너리 등 모든 파이썬 객체를 인수로 넘길 수 있습니다.

- Q. 반복 처리 코드를 함수로 만들어두면 어떤 점이 편리한가요?

정답 확인: 새로운 데이터 리스트가 생길 때마다 똑같은 for 문을 다시 작성할 필요 없이 함수 호출 한 줄로 처리할 수 있습니다.

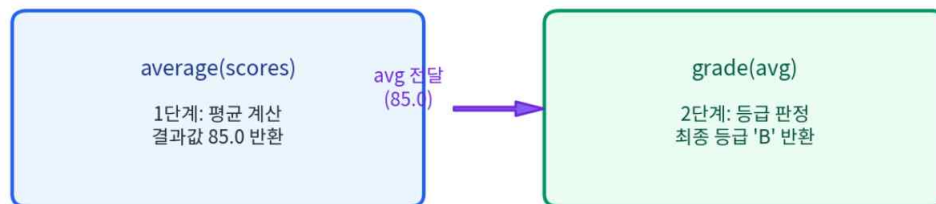
오늘의 핵심 한 문장 : 반복문을 함수에 넣으면 여러 데이터를 반복 처리하는 기능을 재사용할 수 있다.

[9차시] 여러 함수를 연결하기 - 프로그램을 작은 기능으로 나누기

학습 목표: 하나의 큰 문제를 여러 개의 작은 기능별 함수로 분해하고, 한 함수의 반환값을 다른 함수에 파이프라인처럼 연결한다.

1. 개념 이야기: 분업으로 완성하는 햄버거 공장

햄버거를 만들 때 한 사람이 빵도 굽고, 패티도 굽고, 포장까지 혼자 다 하면 너무 비효율적입니다. 패티 담당, 야채 담당, 소스 담당이 각자 맡은 일만 완벽히 끝내고 다음 사람에게 넘겨주는 분업 시스템이 훨씬 빠르고 고장도 적습니다. 프로그래밍도 마찬가지입니다. 평균 계산 함수와 등급 판정 함수처럼 역할을 딱 하나씩만 맡겨서 연결하면 코드 수정과 오류 찾기가 엄청나게 쉬워집니다!



2. 설명용 핵심 예제

```
def average(scores):  
    return sum(scores) / len(scores)  
  
def grade(avg):  
    if avg >= 90:  
        return "A"  
    elif avg >= 80:  
        return "B"  
    elif avg >= 70:  
        return "C"  
    else:  
        return "D"  
  
scores = [85, 92, 78]  
avg = average(scores)  
  
print("평균:", avg)  
print("등급:", grade(avg))
```

3. 코드 한 줄씩 살펴보기

- **average(scores)** 오직 점수들의 평균 계산 작업에만 집중하는 전문 함수입니다.

- **grade(avg)** 평균 숫자를 받아 등급 판정 작업에만 집중하는 전문 함수입니다.
- **grade(avg)** average()가 return해 준 반환값(avg)을 그대로 grade()의 인수로 바통 터치합니다.

4. 실습 도전 과제

- **실습 1.** 섭씨→화씨 변환 함수와, 변환된 온도의 쾌적 상태를 판정하는 함수를 각각 만들고 연결해 보기
- **실습 2.** 신장과 체중으로 BMI를 계산하는 함수와, 그 결과를 받아 비만도를 판정하는 함수를 각각 만들어 연결해 보기

5. 수업 마무리 확인 퀴즈

- **Q. 큰 프로그램을 여러 개의 작은 함수로 나누는 가장 큰 장점 2가지는?**
정답 확인: 1) 코드의 가독성이 높아지고, 2) 문제가 생겼을 때 해당 함수만 수정하면 되므로 디버깅이 쉬워집니다.
- **Q. 한 함수에서 return한 결과값을 다른 함수의 인수로 바로 사용할 수 있나요?**
정답 확인: 네, `print(grade(average(scores)))`처럼 반환값을 다른 함수의 인수로 직접 넘겨줄 수 있습니다.

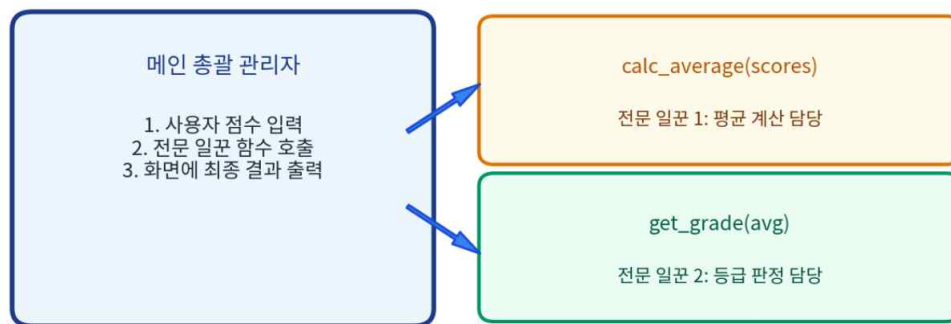
오늘의 핵심 한 문장 : 큰 문제는 역할이 분명한 여러 함수로 나누면 이해하고 수정하기 쉽다.

[10차시] 미니 프로젝트 - 함수로 완성형 성적 관리 프로그램 제작

학습 목표: 문제를 기능 단위로 나누어 설계하고, 매개변수·return·조건문·반복문을 종합하여 독립된 프로그램을 완성한다.

1. 개념 이야기: 건축 설계도와 오케스트라 총괄 지휘

드디어 함수 마스터가 되는 마지막 프로젝트 시간입니다! 프로그래머는 무작정 첫 줄부터 코딩을 시작하지 않습니다. 마치 멋진 건물을 짓기 전 설계도를 그리듯, 어떤 부품 함수들이 필요한가를 먼저 정의합니다. 계산 담당 일꾼, 판정 담당 일꾼, 출력 담당 일꾼을 각각 만들어둔 뒤, 메인 지휘자가 사용자 입력을 받아 일꾼들을 착착 부려 결과물을 만들어냅니다. 이렇게 만든 모듈화 함수들은 뒤이어 배울 Tkinter GUI 버튼에 그대로 쏙 들어가게 됩니다!



2. 프로젝트 통합 핵심 코드

```
def calc_average(scores):  
    return sum(scores) / len(scores)  
  
def get_grade(avg):  
    if avg >= 90:  
        return "A"  
    elif avg >= 80:  
        return "B"  
    elif avg >= 70:  
        return "C"  
    elif avg >= 60:  
        return "D"  
    else:  
        return "F"  
  
def print_result(name, scores):  
    avg = calc_average(scores)  
    print("=" * 25)  
    print("학생:", name)  
    print("평균:", round(avg, 1), "점")  
    print("등급:", get_grade(avg))
```

```

print("=" * 25)

# --- 메인 실행 흐름 ---
name = input("이름: ")
scores = []

for subject in ["국어", "영어", "수학"]:
    score = int(input(subject + " 점수: "))
    scores.append(score)

print_result(name, scores)

```

3. 코드 한 줄씩 살펴보기

- **calc_average():** 평균 계산 전문 부품 함수
- **get_grade():** 등급 판정 전문 부품 함수
- **print_result():** 두 부품을 조율하여 최종 성적표를 예쁘게 출력하는 총괄 출력 함수
- **메인 코드 흐름** 사용자로부터 3과목 점수를 입력받아 리스트에 넣은 뒤 print_result()에 전달하여 프로그램을 완성합니다.

4. 실습 도전 과제

- **실습 1.** 프로젝트 A: 편의점 메뉴 가격과 수량을 입력받아 총액과 할인액을 함수로 계산하는 프로그램 제작
- **실습 2.** 프로젝트 B: 여러 숫자를 입력받아 합계·평균·최댓값을 각각의 함수로 계산해 보여주는 프로그램 제작
- **실습 3.** 프로젝트 C: 3문항의 상식 퀴즈를 풀고 정답 개수에 따라 합격/불합격을 판정하는 퀴즈 프로그램 제작

5. 수업 마무리 확인 퀴즈

- Q. 프로그램을 만들기 전 함수로 분리할 기능들을 먼저 기획하였나요?
정답 확인: 네, 입력, 계산, 판정, 출력을 사전에 나누어 설계했습니다.
- Q. 작성한 함수의 이름만 보고도 어떤 역할을 하는지 직관적으로 알 수 있나요?
정답 확인: calc_average, get_grade처럼 동사+명사 형태로 역할을 명확히 표현했습니다.
- Q. 하나의 함수가 너무 많은 작업(입력과 계산과 출력을 동시에)을 하고 있지는 않은가요?
정답 확인: 한 함수당 단 하나의 핵심 책임만 가지도록 깔끔하게 분리했습니다.

오늘의 핵심 한 문장 : 좋은 프로그램은 필요한 기능을 먼저 나누고, 각 기능을 함수로 구현한다.